

AI 100 講 — 補充單元

Everything Claude Code

10 個月實戰打磨的 Skills + Agents + Memory + Security 框架

按 → 或空白鍵開始

今天的旅程

Everything

Part 1

ECC 是什麼？

不只是設定檔，是完整的 AI 開發效能系統

ECC 的定位



不只是設定檔，是完整的 AI 開發效能系統

ECC 的定位

一般 Claude Code

安裝 ECC
Part 2

開發效能升級

翻牆 → 忘記

Skill, Agent, Method

自動學習、自動審查

四大核心能力

學習、記憶、安全、協作

核心一：持續學習

ECC 自動從每次 session 提取模式，轉化為可重用技能

```
# 3 API error handling
```

```
↓ ECC
```

```
# skill: "api-error-handling-pattern"
```

```
↓
```

```
#
```

學習、記憶、安全、協作

核心一：持續學習

核心二：Memory 持久化

ECC 自動從每次 session 提取模式，轉化為可重用技能

透過 Hooks 自動保存/載入 session 狀態



核心三：AgentShield 安全掃描

透過 Hooks 自動保存/讀取 session 狀態
三層防護架構



核心四：13 個專業 Agent 協作

核心三：AgentShield 安全掃描

三層防護架構

核心四：13 個專業 Agent 協作

Part 3

安裝與結構

3 分鐘安裝，立即提升

```
# 複製
git clone https://github.com/affaan-m/everything-claude-
code.git
cd everything-claude-code

# 安裝
./install.sh python # 安裝 Python
./install.sh typescript # 安裝 TypeScript
./install.sh python typescript # 安裝 Python 和 TypeScript
```

安裝了什麼？

```
~/.claude/rules/
├── common/                # 通用規則
│   ├── agents.md          # Agent 規則
│   ├── coding-style.md    # 編碼風格
│   ├── development-workflow.md
│   ├── git-workflow.md
│   ├── security.md        # 安全規則
│   └── testing.md         # 測試規則
├── python/               # Python 規則
└── typescript/           # TypeScript 規則
```


56+ Skills • 32 Slash Commands

類別	Skills 範例
claude.rules	
	— auto-fix
	— coding-style
	— development-workflow
	— git-workflow
	— security
	— test

56+ Skills • 32 Slash Commands

常用 Slash Commands

類別

Skills 範例

```
/plan "??????"      # ????
/architect "?? API"   # 
/tdd "?????"         # ????
/review              # 
/security-scan        # 
/learn-eval          # 
```

安裝前 vs 安裝後

安裝前	安裝後 (ECC)

支援語言與部署方式

安裝前 vs 安裝後

安裝前

安裝後 (ECC)

支援語言與部署方式
一句話記住

Claude Code 很強

ECC 讓它 越用越強

Skills + Agents + Memory + Security = 完整開發效能系統

github.com/affaan-m/everything-claude-code